

Software Requirements Specification

Smart Residential Society Management System (RSMS)

Version 2.0 · Updated Edition
September 17, 2026

Prepared for: Tanvir · RSMS Build Package

Document status	Updated — reflects the implemented system
Backend stack	NestJS 11 · TypeORM · PostgreSQL · JWT · Zod
Frontend stack	Next.js (App Router) · TypeScript · Tailwind CSS · next-intl
Supersedes	RSMS SRS v1.0 (original) and MASTER_PROMPT.md gap analysis

Document Revision History

Version	Date	Description	Author
1.0	2026-09-16	Original build-prompt specification (RSMS Master Build Prompt), including the initial SRS gap-analysis.	RSMS Project Team
2.0	2026-09-17	Updated to match the delivered system: split Blocks/Flats into distinct modules, documented the Amenities, Notifications, Reports and Profile Picture Management features, refined the API and data model to the shipped schema, and expanded non-functional requirements.	RSMS Project Team

Table of Contents

Document Revision History	2
Table of Contents	3
1. Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Definitions, Acronyms and Abbreviations	6
1.4 References	6
1.5 Document Overview	6
2. Overall Description	7
2.1 Product Perspective	7
2.2 Product Functions Summary	7
2.3 User Classes and Characteristics	7
2.4 Operating Environment	8
2.5 Design and Implementation Constraints	8
2.6 Assumptions and Dependencies	8
3. System Features (Functional Requirements)	9
3.1 Authentication & Account Security	9
3.2 User Management	9
3.3 Residents, Blocks & Flats	10
3.4 Complaint Management	10
3.5 Visitor Management	11
3.6 Billing & Payments	11
3.7 Announcements	12
3.8 Amenities & Bookings	12
3.9 Notifications	12
3.10 Reports & Dashboard	13
3.11 Profile Management	13
4. External Interface Requirements	13
4.1 User Interfaces	13

4.2 API Interfaces	13
4.3 Hardware Interfaces	14
4.4 Communications Interfaces	14
5. Non-Functional Requirements	15
5.1 Security	15
5.2 Performance	15
5.3 Usability & Accessibility	15
5.4 Reliability & Availability	15
5.5 Maintainability	16
5.6 Portability & Deployment	16
5.7 Localization	16
6. System Architecture	16
6.1 Architectural Style	16
6.2 Backend Request Pipeline	16
6.3 Technology Stack	17
7. Data Model	17
7.1 Core Entities	17
7.2 Role Permission Matrix	19
Appendix A — Environment Configuration	19
Appendix B — Glossary	20
Appendix C — Explicitly Out of Scope	20

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the Smart Residential Society Management System (RSMS). It supersedes the original RSMS specification and the accompanying gap-analysis captured in *MASTER_PROMPT.md*, and instead documents the system as it has actually been designed and built: a NestJS/PostgreSQL backend exposing a versioned REST API, and a Next.js frontend consuming it. It is intended to be the single reference point for what the system does today, for stakeholders, developers extending the platform, and QA teams validating it.

1.2 Scope

RSMS is a role-governed web platform that digitizes the day-to-day administration of a residential apartment complex in place of paper-based or spreadsheet-based workflows. The system covers:

- Account management and role-based access control for five user types.
- Resident onboarding and flat/block occupancy tracking.
- Maintenance complaint submission, assignment and lifecycle tracking.
- Visitor pre-registration, gate verification, and resident-approved walk-ins.
- Monthly billing generation, defaulter tracking, and simulated online payment.
- Society-wide and targeted announcements.
- Shared amenity scheduling and booking.
- In-app and e-mail notifications.
- Operational reporting (PDF/Excel export) and a manager KPI dashboard.
- Self-service profile management, including profile picture upload.

Explicitly **out of scope** for this version: SMS delivery, live/real payment gateway integration (bKash/Nagad/card payments are simulated), push notifications, and any cloud-hosting or containerisation concerns — see Section 2.5 for the full constraint list.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
RSMS	Smart Residential Society Management System — the product described by this document.
SRS	Software Requirements Specification.
FR	Functional Requirement.
NFR	Non-Functional Requirement.
RBAC	Role-Based Access Control.
JWT	JSON Web Token — used for stateless API authentication.
OTP	One-Time Password — a 6-digit, time-limited code e-mailed for password resets.
Flat	An individual residential unit, located on a floor within a block.
Block	A building or wing within the society containing multiple floors and flats.
Defaulter	A resident with one or more bills past their due date and unpaid.
Manager / Guard / Maintenance / Accountant / Resident	The five user roles recognised by the system (Section 2.3).

1.4 References

- *RSMS — Master Build Prompt* (MASTER_PROMPT.md) — original specification and SRS gap analysis.
- *AI Coding Assistant — Build Prompt* (AI_BUILD_PROMPT.md) — implementation build order and coding conventions.
- *RSMS API Endpoint Reference* (README/APIEndpoint.md) — full REST contract with request/response examples.
- Interactive API documentation — Swagger UI, served at `/api/docs` on the running backend.

1.5 Document Overview

Section 2 describes the product at a high level. Section 3 enumerates functional requirements per module, each with a stable identifier for traceability. Section 4 covers external interfaces. Section 5 defines non-functional requirements. Section 6 describes the system architecture, Section 7 the data model and permission matrix, and the appendices contain configuration and glossary reference material.

2. Overall Description

2.1 Product Perspective

RSMS is a new, self-contained product rather than an extension of an existing system. It follows a conventional two-tier web architecture: a Next.js single-page application (the client) communicates over HTTPS/JSON with a NestJS REST API (the server), which is the sole owner of a PostgreSQL database. The backend also sends transactional e-mail via Gmail SMTP (Nodemailer) for OTPs, welcome messages, and status/billing notifications. No other external systems are integrated; all payment and SMS interactions in the current version are simulated (see 2.5).

2.2 Product Functions Summary

Module	Summary
Authentication & Security	Login, JWT access/refresh tokens, account lockout, e-mail OTP password reset.
User Management	Manager-driven CRUD for all account types; self-service password and profile-picture changes.
Residents & Occupancy	Resident onboarding, block/flat inventory, assignment and vacancy tracking, occupancy history.
Complaints	Resident-submitted maintenance tickets with category/priority, assignment, status lifecycle, reopen, and reporting.
Visitors	Pre-registration, gate search and verification, walk-in approval workflow, suspicious-visitor flagging.
Billing & Payments	Configurable per-flat-type billing, monthly bill generation, simulated payment, PDF receipts, defaulter tracking, reminders.
Announcements	Society-wide or block/floor-targeted notices with in-app and e-mail delivery.
Amenities	Amenity and slot configuration, conflict-free booking, dues-based booking restriction, cancellation.
Notifications	Per-user in-app notification feed with unread-count polling and read/unread state.
Reports	Manager/Accountant KPI dashboard and downloadable PDF/Excel operational reports.
Profile Management	Self-service profile view, password change, and profile picture upload/removal for every role.

2.3 User Classes and Characteristics

RSMS recognises exactly five roles, enforced by a role-based access guard on every protected endpoint:

Role	Description	Primary responsibilities
Resident	An owner or tenant occupying a flat.	Submit complaints, pre-register visitors, view/pay bills, book amenities, view announcements.
Manager	Society administrator with the broadest access.	Manage user accounts, residents, blocks/flats, assign complaints, publish announcements, configure amenities, view reports.
Guard	Front-gate security personnel.	Search pre-registrations, log visitor entry/exit, log walk-ins, flag suspicious visitors.

Role	Description	Primary responsibilities
Maintenance	Facilities/maintenance staff.	Work assigned complaints, update status, record work notes.
Accountant	Finance/billing administrator.	Configure billing rules, generate monthly bills, track defaulters, send reminders, export billing reports.

All roles share a common account model (single users table with a role enum) — there is no separate inheritance hierarchy for staff types. Every authenticated user, regardless of role, can view and edit their own profile, change their password, and manage their own profile picture.

2.4 Operating Environment

- **Server:** Node.js runtime hosting the NestJS API, connected to a PostgreSQL 14+ database.
- **Client:** Any evergreen desktop or mobile web browser (Chrome, Edge, Firefox, Safari) rendering the Next.js application; no native app is required.
- **E-mail delivery:** Gmail SMTP via an app password, configured through environment variables.
- **File storage:** Uploaded profile pictures are written to local disk under the backend's uploads/ directory and served as static files; no third-party object storage is used.

2.5 Design and Implementation Constraints

The following constraints were fixed for this build and intentionally narrow the original specification's scope:

Area	Constraint applied
Validation	Zod schemas validate every request body; class-validator/class-transformer are not used.
Authentication	Stateless JWT access tokens (15 min) plus a rotated, DB-persisted refresh token (7 days) — no server-side session store.
Notifications	In-app notifications plus e-mail only. No SMS provider, no push notifications (APNs/FCM), no WebSockets.
Payments	bKash/Nagad/Card payment is a simulated flow: any transaction reference is accepted and the bill is marked paid; no live gateway is called.
Infrastructure	No AWS/cloud SDK, no Docker, no Redis or other cache layer, in this version.
File uploads	Local disk storage for profile pictures (JPEG/PNG/WEBP, ≤2 MB); not yet backed by shared/object storage, which matters if the API is scaled across multiple instances.
Internationalisation	English and Bangla only, selected via a locale-prefixed URL and a topbar switcher.

2.6 Assumptions and Dependencies

- A single society/complex is managed per deployment; multi-tenancy across societies is not modelled.
- The Manager role is trusted to correctly configure billing rules, blocks, and flats before residents are onboarded.
- Gmail SMTP credentials (an app password) are available and not rate-limited beyond the society's expected e-mail volume.
- Users have a modern browser with JavaScript enabled; no support is provided for legacy browsers.

3. System Features (Functional Requirements)

Requirements are grouped by module and identified as FR-<module>-<n> for traceability. Priority follows MoSCoW: **M** = Must have, **S** = Should have, **N** = Nice to have. Unless stated otherwise, every write endpoint validates its payload with a Zod schema and returns the standard error envelope described in Section 4.2.

3.1 Authentication & Account Security

ID	Requirement	Priority
FR-AUTH-01	The system shall authenticate a user by email and password and, on success, issue a short-lived JWT access token (15 minutes) and a rotating refresh token (7 days, persisted server-side).	M
FR-AUTH-02	The system shall lock an account for 1 hour after 3 consecutive failed login attempts, and reset the failed-attempt counter on a successful login.	M
FR-AUTH-03	The system shall allow a client to exchange a valid, unrevoked refresh token for a new access/refresh pair, revoking the previous refresh token (rotation).	M
FR-AUTH-04	The system shall allow a user to revoke their refresh token on logout.	M
FR-AUTH-05	The system shall let any user request a password reset by e-mail; it shall generate a 6-digit OTP valid for 10 minutes and send it by e-mail, without revealing whether the e-mail address is registered.	M
FR-AUTH-06	The system shall allow a user to verify an OTP's validity and, if valid, set a new password; on success it shall revoke all of that user's existing refresh tokens.	M
FR-AUTH-07	The system shall expose an endpoint returning the profile of the currently authenticated user.	M
FR-AUTH-08	The system shall reject requests to protected endpoints that lack a valid, unexpired JWT, and requests where the authenticated user's role does not match the endpoint's required role(s).	M

3.2 User Management

ID	Requirement	Priority
FR-USR-01	The Manager shall be able to create a user account of any role; if no password is supplied, the system shall generate one and e-mail it via a welcome message.	M
FR-USR-02	The Manager shall be able to list all user accounts with pagination and an optional role filter.	M
FR-USR-03	The Manager shall be able to view, update (name/phone), deactivate, and re-activate any user account; re-activation shall also clear any lockout state.	M
FR-USR-04	Any authenticated user shall be able to change their own password after confirming their current password.	M
FR-USR-05	Any authenticated user shall be able to upload a profile picture (JPEG, PNG or WEBP, ≤2 MB); a new upload shall replace and delete the previous image.	S
FR-USR-06	Any authenticated user shall be able to remove their profile picture, reverting their profile to initials-based display.	S

3.3 Residents, Blocks & Flats

ID	Requirement	Priority
FR-RES-01	The Manager shall be able to onboard a resident in a single operation that creates both the underlying user account (role RESIDENT) and the resident profile (type OWNER/TENANT, emergency contact, family members, move-in date).	M
FR-RES-02	The Manager shall be able to list all residents together with their linked flat and block.	M
FR-RES-03	A Resident shall be able to view and update their own profile (emergency contact, family members); a Manager may update any resident's profile.	M
FR-RES-04	The Manager shall be able to view the occupancy history of a given flat.	S
FR-BLK-01	The Manager shall be able to create and list blocks (buildings/wings) that group flats.	M
FR-FLT-01	The Manager shall be able to create a flat under a block with floor number, flat number, area and flat type.	M
FR-FLT-02	The Manager (and Guard, for gate-terminal lookups) shall be able to list all flats and view a single flat's detail including its current resident(s).	M
FR-FLT-03	The Manager shall be able to assign a resident to a flat (marking it occupied) and to vacate a flat (marking it vacant and closing the resident's occupancy record with a move-out date).	M

3.4 Complaint Management

ID	Requirement	Priority
FR-CMP-01	A Resident shall be able to submit a complaint with a title, description, category (Electrical, Plumbing, Civil, Pest Control, Other) and priority (Low, Medium, High, Critical; default Medium).	M
FR-CMP-02	The system shall support the complaint status lifecycle Pending → Assigned → In Progress → Resolved → Closed, with Resolved/Closed complaints reopenable by the owning Resident (→ Reopened).	M
FR-CMP-03	A Manager shall be able to assign a Pending complaint to a Maintenance-role user.	M
FR-CMP-04	Maintenance staff and Managers shall be able to update a complaint's status; Maintenance staff shall be able to attach free-text work notes.	M
FR-CMP-05	A Resident shall be able to view their own complaints; Maintenance staff shall be able to view complaints assigned to them; a Manager shall be able to view and filter all complaints by status and category.	M
FR-CMP-06	The system shall provide a summary report of complaints grouped by status, category and assigned staff member, optionally filtered by a date range.	S
FR-CMP-07	A status change on a complaint shall trigger an in-app notification and e-mail to the owning resident.	M

3.5 Visitor Management

ID	Requirement	Priority
FR-VIS-01	A Resident shall be able to pre-register an expected visitor with name, phone, purpose, visitor type (Regular, Delivery, Suspicious) and expected arrival time.	M
FR-VIS-02	A Guard shall be able to search pre-registered visitors by name or phone at the gate.	M
FR-VIS-03	A Guard shall be able to log a visitor's entry (auto-approving pre-registered visitors) and later log their exit.	M
FR-VIS-04	A Guard shall be able to log an unregistered walk-in visitor against a target flat; the system shall notify every resident of that flat for approval.	M
FR-VIS-05	A Resident shall be able to approve or deny a walk-in visitor logged against their flat; if there is no resident response, the Guard may proceed at their discretion (operational fallback, not system-enforced timeout).	M
FR-VIS-06	A Guard shall be able to flag any visitor as suspicious, with an optional reason.	S
FR-VIS-07	Managers and Guards shall be able to view the full visitor log and a delivery-only view of it.	S
FR-VIS-08	The system shall notify a resident when their pre-registered visitor arrives at the gate.	S

3.6 Billing & Payments

ID	Requirement	Priority
FR-BIL-01	An Accountant shall be able to define/update a billing configuration per flat type (base amount, extra vehicle charge, commercial surcharge, late-penalty percentage, billing day).	M
FR-BIL-02	An Accountant shall be able to generate bills for a given month/year; the system shall create one bill per active resident whose flat type has a matching configuration, skipping residents who already have a bill for that period.	M
FR-BIL-03	A Resident shall be able to view their own bills; an Accountant shall be able to list and filter all bills by status (Pending, Paid, Overdue, Partial).	M
FR-BIL-04	A Resident shall be able to pay a bill via a simulated payment (method: bKash, Nagad or Card, with a user-supplied transaction reference); the system shall record the payment and mark the bill Paid without contacting any real payment gateway.	M
FR-BIL-05	A Resident shall be able to download a PDF receipt for a paid bill.	S
FR-BIL-06	The system shall identify bills past their due date and not fully paid as Overdue (defaulters), viewable by Accountants and Managers.	M
FR-BIL-07	An Accountant shall be able to trigger reminder notifications (in-app + e-mail) to every resident with an overdue bill.	S
FR-BIL-08	The system shall provide an Accountant-facing financial dashboard summarising total billed, total collected, pending count and overdue count.	S

3.7 Announcements

ID	Requirement	Priority
FR-ANN-01	A Manager shall be able to publish an announcement with a title, body, optional attachment URL, and a scope of All, Block, or Floor.	M
FR-ANN-02	When the scope is Block or Floor, the Manager shall specify one or more targets (block and/or floor number); the system shall create the corresponding target records.	M
FR-ANN-03	A Resident shall see All-scope announcements plus any announcement targeted at their own block/floor; Managers and staff shall see all announcements.	M
FR-ANN-04	A Manager shall be able to edit or delete an announcement they manage.	S
FR-ANN-05	Publishing a targeted or society-wide announcement shall create an in-app notification and send an e-mail to every targeted resident.	M

3.8 Amenities & Bookings

ID	Requirement	Priority
FR-AMN-01	A Manager shall be able to create an amenity (name, description, capacity, location) together with a set of recurring weekly time slots.	M
FR-AMN-02	A Manager shall be able to update an amenity's configuration, including activating/deactivating it.	S
FR-AMN-03	Any authenticated user shall be able to list active amenities with their available slots; a Resident shall be able to view slot availability for a specific date.	M
FR-AMN-04	A Resident shall be able to book an available slot for a specific date; the system shall reject the booking with a 403 if the same slot/date is already booked, or if the resident has any Overdue bill.	M
FR-AMN-05	A Resident shall be able to view and cancel their own bookings; a Manager shall be able to view all bookings.	S

3.9 Notifications

ID	Requirement	Priority
FR-NOT-01	The system shall record an in-app notification (title, body, type, reference) for a user whenever a relevant event occurs (complaint status change, visitor arrival, bill generated/overdue, announcement, walk-in approval request).	M
FR-NOT-02	Any authenticated user shall be able to list their own notifications, paginated, and retrieve an unread count for the notification-bell badge.	M
FR-NOT-03	Any authenticated user shall be able to mark a single notification, or all of their notifications, as read.	M
FR-NOT-04	The frontend shall poll the unread-count endpoint periodically to keep the notification badge current.	S

3.10 Reports & Dashboard

ID	Requirement	Priority
FR-RPT-01	The system shall provide a Manager-facing KPI dashboard summarising open complaints, overdue bills, today's visitors, and active bookings.	S
FR-RPT-02	A Manager shall be able to export a complaint summary report in PDF or Excel format.	S
FR-RPT-03	An Accountant shall be able to export a billing report in PDF or Excel format.	S
FR-RPT-04	A Manager shall be able to export a visitor log report in PDF or Excel format.	N

3.11 Profile Management

ID	Requirement	Priority
FR-PRO-01	Every authenticated user shall be able to view their own profile page, showing name, email, phone, role and last-login time.	M
FR-PRO-02	The profile page and the application topbar shall display the user's uploaded profile picture when present, falling back to initials derived from their full name otherwise.	S
FR-PRO-03	Profile picture uploads shall be validated for file type (JPEG/PNG/WEBP only) and size (≤ 2 MB) both in the browser and on the server, with a clear error message on rejection.	M

4. External Interface Requirements

4.1 User Interfaces

The frontend is a Next.js (App Router) single-page application styled with Tailwind CSS. Every route is prefixed with a locale segment (`/en/...` or `/bn/...`); a topbar language switcher toggles between English and Bangla using next-intl. Authenticated routes live under `/dashboard` behind a role-aware sidebar that only renders navigation items the current role is permitted to use. Every form uses React Hook Form with a Zod resolver for client-side validation, and react-hot-toast for success/error feedback.

4.2 API Interfaces

The backend exposes a single versioned REST API under `/api/v1`, documented interactively via Swagger UI at `/api/docs` and in full in `README/APIEndpoint.md`. All protected endpoints require an `Authorization: Bearer <accessToken>` header. Every error response follows a single standardised envelope:

```
{
  "statusCode": 400,
  "message": "Human readable message",
  "errors": { "fieldName": ["error message"] },
  "timestamp": "2026-09-17T10:00:00.000Z",
  "path": "/api/v1/auth/login"
}
```

The API surface comprises twelve resource groups:

Base path	Purpose
/auth	Login, token refresh, logout, forgot/verify/reset password, current-user profile.
/users	Account CRUD (Manager), own-password change, own-avatar upload/removal.
/residents	Resident onboarding, listing, profile view/update, occupancy history.
/blocks	Block inventory management.
/flats	Flat inventory, assignment and vacancy management.
/complaints	Complaint submission, assignment, status/notes, reopen, reporting.
/visitors	Pre-registration, gate search, entry/exit logging, walk-in approval, flagging.
/billing	Billing configuration, bill generation, payment, receipts, defaulters, reminders, dashboard.
/announcements	Announcement CRUD with block/floor targeting.
/amenities	Amenity/slot configuration, booking, cancellation.
/notifications	In-app notification feed, unread count, read/read-all.
/reports	KPI dashboard and PDF/Excel report downloads.

4.3 Hardware Interfaces

None. RSMS is a standard web application; the only device requirement is a browser-capable computer, tablet, or smartphone. No dedicated turnstile, RFID, or camera hardware is integrated in this version — gate verification is performed manually by Guard staff through the web UI.

4.4 Communications Interfaces

The backend communicates with an SMTP relay (Gmail, via Nodemailer) over TLS on port 587 to deliver all transactional e-mail: welcome messages, OTPs, complaint status updates, visitor arrival notices, bill/reminder notices, and announcement broadcasts. All client-server traffic uses HTTP(S) with JSON payloads, except file uploads (profile pictures), which use `multipart/form-data`.

5. Non-Functional Requirements

5.1 Security

ID	Requirement	Priority
NFR-SEC-01	Passwords shall be hashed with bcrypt at a cost factor of 12 before storage; plaintext passwords are never persisted or logged.	M
NFR-SEC-02	Access tokens shall expire after 15 minutes; refresh tokens shall expire after 7 days, be persisted server-side, and be rotated (old token revoked) on every use.	M
NFR-SEC-03	Every protected endpoint shall enforce both authentication (valid JWT) and authorization (role membership) before executing business logic.	M
NFR-SEC-04	An account shall be locked for 1 hour after 3 consecutive failed login attempts, mitigating password-guessing attacks.	M
NFR-SEC-05	All incoming request bodies shall be validated against a Zod schema before reaching business logic; malformed input shall be rejected with a 400 and field-level error detail.	M
NFR-SEC-06	Profile-picture uploads shall be restricted by MIME type and size on the server, independent of client-side checks, to prevent arbitrary file upload.	M
NFR-SEC-07	Security-relevant actions shall be captured in an audit log (user, action, entity, entity ID, timestamp) for traceability.	S

5.2 Performance

- List endpoints (users, residents, complaints, visitors, bills) shall support server-side pagination to keep individual responses small as data volume grows.
- The notification-unread-count endpoint is designed for lightweight, frequent polling (every 30 seconds) from the client without a noticeable server load impact.
- Interactive pages should render their primary content within 2 seconds on a broadband connection under typical society-scale data volumes (hundreds, not millions, of records).

5.3 Usability & Accessibility

- The UI shall be fully available in both English and Bangla, switchable at any time without losing navigation context.
- Every form shall present inline, field-level validation errors in addition to a toast summary for server-side failures.
- Role-based navigation shall only present actions a user is permitted to perform, reducing confusion and accidental permission errors.

5.4 Reliability & Availability

- A failed outbound e-mail (e.g., SMTP timeout) shall be logged but shall not fail the triggering business operation (e.g., bill generation still succeeds even if the notification e-mail fails).
- Token refresh shall be transparent to the user: a 401 response triggers an automatic silent refresh-and-retry before falling back to a login redirect.

5.5 Maintainability

- Backend modules follow a consistent module/controller/service/DTO structure per feature, documented in `AI_BUILD_PROMPT.md`, to keep new modules predictable to add.
- All API endpoints are self-documenting via Swagger/OpenAPI decorators, keeping interactive documentation in sync with the implementation.
- Database schema changes are applied through TypeORM entity definitions; in production, migrations should replace the development-time auto-sync.

5.6 Portability & Deployment

- The backend and frontend are independently deployable Node.js processes, configured entirely through environment variables (see Appendix A) with no hard-coded secrets or endpoints.
- The current file-storage approach (local disk for profile pictures) assumes a single backend instance or a shared volume; horizontally scaling the API across multiple nodes would require migrating to shared/object storage.

5.7 Localization

All user-facing strings are externalised into locale message bundles (English and Bangla); no display text is hard-coded in components. Dates, currency (BDT) and numbers are formatted per-locale on the frontend.

6. System Architecture

6.1 Architectural Style

RSMS follows a layered client-server architecture. The Next.js client renders server and client components, holds the access token in memory (Zustand store, not localStorage) and the refresh token in an httpOnly cookie, and talks to the API exclusively through a typed Axios client with request/response interceptors. The NestJS backend is organised into feature modules (one per domain: auth, users, residents, blocks, flats, complaints, visitors, billing, announcements, amenities, notifications, reports, mail), each following the same controller → service → TypeORM-repository → PostgreSQL layering.

6.2 Backend Request Pipeline

Every incoming request passes through the following stages, in order:

- Global JWT authentication guard (bypassed only for explicitly public routes such as login and OTP endpoints).
- Role guard, evaluating the endpoint's declared `@Roles()` requirement against the authenticated user.
- Zod validation pipe, parsing and validating the request body/query against the route's schema.
- Controller → Service → TypeORM repository, executing business logic and persistence.
- Audit-log interceptor, recording security-relevant actions after the handler completes.
- Global HTTP exception filter, converting any thrown error into the standardised error envelope (Section 4.2).

6.3 Technology Stack

Layer	Technology
Backend framework	NestJS 11 (TypeScript)
ORM / database	TypeORM · PostgreSQL
Authentication	JWT (access + refresh) via @nestjs/jwt and Passport
Validation	Zod, via a custom ZodValidationPipe
API documentation	OpenAPI / Swagger UI (@nestjs/swagger)
E-mail	Nodemailer over Gmail SMTP
File uploads	Multer (disk storage), served as static assets
Reporting exports	PDF and Excel generation for billing/complaint/visitor reports
Frontend framework	Next.js (App Router), TypeScript
Styling	Tailwind CSS
Forms & validation	React Hook Form + Zod resolvers
State management	Zustand (auth/session state) · TanStack Query (server state)
HTTP client	Axios with auth/refresh interceptors
Internationalisation	next-intl (English, Bangla)

7. Data Model

7.1 Core Entities

All entities share a common base (UUID primary key, createdAt, updatedAt). Relationships are enforced at the database level via foreign keys.

Entity	Key attributes	Relationships
User	fullName, email (unique), passwordHash, role, phone, profilePicture, isActive, loginAttempts, lockedUntil, lastLoginAt	1–1 with Resident (when role = RESIDENT)
Block	name, description	1–many Flat
Flat	floorNumber, flatNumber, area, flatType, isOccupied	many–1 Block; 1–many Resident (over time)
Resident	type (Owner/Tenant), emergencyContact, familyMembers (JSON), moveInDate, moveOutDate, isActive	1–1 User; many–1 Flat; 1–many Complaint, Visitor(pre-reg), Bill, Booking
Complaint	category, priority, status, title, description, staffNotes, resolvedAt	many–1 Resident; many–1 User (assignedTo)

Entity	Key attributes	Relationships
Visitor	visitorName, phone, purpose, visitorType, expectedArrival, entryTime, exitTime, verificationStatus, isFlagged	many–1 Resident (pre-registered by); many–1 User (verified by Guard); many–1 Flat
Bill	month, year, baseAmount, extraCharges, latePenalty, totalAmount, dueDate, status	many–1 Resident; 1–many Payment
Payment	amount, method, transactionRef, isPaid, paidAt, receiptUrl	many–1 Bill; many–1 Resident
BillingConfig	flatType, baseAmount, extraVehicleCharge, commercialSurcharge, latePenaltyPercent, billingDay	many–1 User (updatedBy)
Amenity	name, description, capacity, location, isActive	1–many AmenitySlot, Booking
AmenitySlot	dayOfWeek, startTime, endTime, isAvailable	many–1 Amenity; 1–many Booking
Booking	bookingDate, status	many–1 Resident, Amenity, AmenitySlot
Announcement	title, body, scope (All/Block/Floor), attachmentUrl, publishedAt	many–1 User (publishedBy); 1–many AnnouncementTarget
Announcement Target	blockId (nullable), floorNumber (nullable)	many–1 Announcement; many–1 Block
Notification	title, body, type, referenceId, isRead	many–1 User
AuditLog	action, entity, entityId, ipAddress, userAgent, meta (JSON)	many–1 User
OtpToken	token, purpose, expiresAt, usedAt	many–1 User
RefreshToken	token, expiresAt, isRevoked	many–1 User

7.2 Role Permission Matrix

Capability	Resident	Manager	Guard	Maintenance	Accountant
View / edit own profile & avatar	✓	✓	✓	✓	✓
Manage user accounts	—	✓	—	—	—
Submit / reopen complaint	✓	—	—	—	—
Assign complaint	—	✓	—	—	—
Update complaint status / notes	—	✓	—	✓	—
Pre-register visitor / approve walk-in	✓	—	—	—	—
Log visitor entry / exit / walk-in	—	—	✓	—	—
Configure billing / generate bills	—	—	—	—	✓
Pay bill (simulated)	✓	—	—	—	—
View defaulters / send reminders	—	✓	—	—	✓
Publish announcement	—	✓	—	—	—
Book amenity	✓	—	—	—	—
Configure amenities	—	✓	—	—	—
View KPI dashboard / reports	—	✓	—	—	✓ (billing)

Appendix A — Environment Configuration

Both applications are configured entirely through environment variables; no secrets are hard-coded.

Backend (.env)

Variable	Purpose
NODE_ENV, PORT, API_PREFIX	Runtime mode, listen port, and API path prefix (api/v1).
DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD, DB_NAME	PostgreSQL connection.
JWT_SECRET, JWT_ACCESS_EXPIRY, JWT_REFRESH_EXPIRY	Token signing secret and lifetimes.
MAIL_HOST, MAIL_PORT, MAIL_USER, MAIL_PASS, MAIL_FROM	Gmail SMTP credentials for outbound e-mail.
FRONTEND_URL	Allowed CORS origin for the web client.
OTP_EXPIRY_MINUTES	Password-reset OTP validity window (default 10).
ACCOUNT_LOCKOUT_ATTEMPTS, ACCOUNT_LOCKOUT_DURATION_HOURS	Login lockout policy (default 3 attempts / 1 hour).
LATE_PENALTY_PERCENT, BILLING_DAY	Default billing behaviour.

Frontend (.env.local)

Variable	Purpose
NEXT_PUBLIC_API_URL	Base URL of the backend API (e.g. <code>http://localhost:3001/api/v1</code>).
NEXT_PUBLIC_APP_NAME	Display name used in the UI shell.

Appendix B — Glossary

Term	Meaning
Walk-in visitor	A visitor who arrives without a prior pre-registration and must be approved by the target resident at (or shortly after) arrival.
Defaulter	A resident whose bill has passed its due date without full payment; the bill's status is automatically set to Overdue.
Scope (announcement)	The audience of an announcement: All residents, a specific Block, or a specific Floor.
Slot	A recurring weekly time window (day of week, start/end time) during which an amenity can be booked.
Token rotation	Issuing a new refresh token and revoking the previous one every time it is used, limiting the value of a stolen refresh token.

Appendix C — Explicitly Out of Scope

The following were present in earlier drafts of the specification and were deliberately excluded from this build:

- SMS-based two-factor authentication or notifications — replaced entirely by e-mail OTP and e-mail notifications.
- Live bKash/Nagad/card payment gateway integration — payments are simulated for demonstration purposes.
- Push notifications (mobile/browser) — replaced by in-app notifications plus e-mail.
- Cloud/object storage, Docker, and container orchestration — the reference deployment runs as plain Node.js processes against a local/managed PostgreSQL instance.
- Real-time transport (WebSockets/Socket.io) — the notification bell uses periodic polling instead.